

Raspodijeljeni sustavi, Predavanje 03

Komunikacija i udaljeni pozivi

Fakultet informatike, Sveučilište Jurja Dobrile u Puli

izv. prof. dr. sc. Nikola Tanković

1 Ishodi učenja

Nakon ovog predavanja trebali biste moći:

1. Objasniti po čemu se udaljeni poziv razlikuje od lokalnog i navesti tri moguća ishoda zahtjeva: uspjeh, pogrešku i nepoznat ishod.
2. Opisati tijek poziva udaljene procedure (RPC): ulogu stuba, serijalizacije i ugovora sučelja, na primjeru gRPC-a.
3. Usporediti JSON i Protocol Buffers te promijeniti ugovor tako da postojeći klijenti nastave raditi.
4. Odabrati rok čekanja i objasniti zašto se rok prenosi kroz lanac usluga.
5. Objasniti zašto ponovni pokušaj udaljenog zahtjeva može udvostručiti učinak te razlikovati semantike *najviše jednom*, *barem jednom* i *učinak točno jednom*.
6. Učiniti operaciju sigurnom za ponavljanje: idempotentnošću, ključem idempotentnosti i atomarnim spremanjem ključa i rezultata.

Predznanje: predavanje 01 (istek vremena čekanja, ponovni pokušaj, tri povijesti istog isteka čekanja, invarijanta, idempotentnost iz kolegija Web aplikacije) i predavanje 02 (granica usluge, podjela portala na usluge, sinkroni i asinkroni pozivi).

Opseg predavanja: zahtjev i odgovor, RPC, gRPC i Protocol Buffers, serijalizacija i promjena ugovora, rokovi čekanja, ponovni pokušaji i idempotentnost. Kako klijent pronalazi adresu usluge obrađuje predavanje 04, odgodu između pokušaja (backoff) i otkrivanje ispada predavanje 08, transakcije preko više usluga predavanje 09, a redove poruka predavanje 10. Portal za predaju projekata i dalje je zamišljeni primjer, a ne stvarni sustav sveučilišta.

2 Primjer: snimanje predaje ne odgovara

U predavanju 02 podijelili smo portal na usluge. Kada student pošalje projekt, portal (web aplikacija s kojom razgovara preglednik) pozove **uslugu predaje** da predaju snimi: usluga zapiše predaju u svoju tablicu `predaje` i vrati identifikator zapisa. To je kratak udaljeni poziv funkcije `SnimiPredaju`. Na zapis predaje oslanja se sve ostalo: nastavnik vidi predaju na popisu, pokreće se provjera projekta, a nastavnik dobiva obavijest.

Večer prije roka portal pozove `SnimiPredaju` za predaju `p-2026-0812` i čeka najviše 2 sekunde. Odgovor ne stigne. Je li predaja snimljena? Smije li portal pozvati ponovno?

Pitanje za ovo predavanje: smije li portal ponoviti poziv? Odgovori „da, odmah", „ne, nikad" i „ovisi" vode u vrlo različite sustave. Ako ponovi, a prvi poziv je ipak stigao, u tablici nastaju **dva zapisa iste predaje**: nastavnik vidi dvije predaje, provjera se pokreće dvaput i stižu dvije obavijesti. Ako ne ponovi, a prvi poziv se izgubio, predaja nije snimljena, a student misli da je predao. Ovo je invarijanta iz predavanja 01: jedna predaja, najviše jedan prihvaćeni zapis. Do kraja predavanja odgovor ćemo obrazložiti.

2.1 Za početak: obavijest poslana dvaput

Predavanje 02 završilo je zadatkom: nastavnik želi e-poštu kada stigne predaja, a usluga e-pošte ponekad satima ne radi. Razumno rješenje je zasebna usluga obavijesti kojoj usluga predaje nakon prihvaćanja šalje asinkronu poruku, pa predaja ne ovisi o e-pošti.

Treće pitanje glasilo je: što ako se ista obavijest pošalje dvaput? Do toga dolazi kada usluga predaje ne dobije potvrdu da je poruka primljena pa je pošalje ponovno. Za obavijest je dvostruka e-pošta neugodna, ali uglavnom bezopasna. Isti obrazac kod naplate, upisa ocjene ili snimanja predaje nije bezopasan. Ako poruka nosi identifikator predaje, usluga obavijesti može prepoznati da je obavijest za tu predaju već poslala. Danas razrađujemo upravo tu ideju: kada je ponavljanje bezopasno i kako ga takvim učiniti.

3 Udaljeni poziv nije lokalni poziv

Lokalni poziv funkcije i poziv funkcije u drugom procesu, na drugom računalu, u kodu mogu izgledati gotovo jednako. Ponašaju se vrlo različito.

Svojstvo	**Lokalni poziv**	**Udaljeni poziv**
Trajanje	nanosekunde do mikrosekunde	milisekunde ili više; ovisi o mreži i drugoj usluzi
Ishod	funkcija vrati vrijednost ili baci iznimku	uspjeh, pogreška ili **nepoznat ishod**
Argumenti	prenosi se referenca na istu memoriju	kopija pretvorena u bajtove i poslana mrežom

Svojstvo	Lokalni poziv	Udaljeni poziv
Ispad	pozivatelj i pozvana funkcija padaju zajedno	mogu ispasti neovisno: djelomični ispad
Ponavljanje	nije potrebno	ponekad nužno, a ponekad opasno

Tablica okuplja ono što smo već vidjeli. U predavanju 02 udaljeni poziv bio je oko 5000 puta sporiji od pristupa memoriji. U predavanju 01 isti istek čekanja mogao je značiti tri različite povijesti. Kleppmann ističe da RPC okviri desetljećima pokušavaju olakšati programiranje raspodijeljenih sustava, ali da su temeljni problemi ostali isti [1, odjeljak 1.3]; pritom upućuje na poznati rad Walda i suradnika iz 1994. godine.

3.1 Zahtjev i odgovor

Definicija 3.1

U komunikaciji **zahtjev–odgovor (request–response)** klijent šalje poruku zahtjeva i čeka poruku odgovora koja se na taj zahtjev odnosi.

Najpoznatiji primjer je HTTP. Kada preglednik pošalje projekt usluzi predaje, kod u JavaScriptu mora razlikovati tri ishoda [1, odjeljak 1.3]:

```
try {
  const odgovor = await fetch("/predaje", {
    method: "POST", body: zip,
    signal: AbortSignal.timeout(30_000),   // čekamo najviše 30 s
  });
  if (odgovor.ok) prikaziPotvrdu(await odgovor.json()); // 1. uspjeh
  else prikaziPogresku(odgovor.status);       // 2. pogreška poslužitelja
} catch (e) {
  prikaziNepoznato();                // 3. nema odgovora: ishod nepoznat
}
```

Prva dva ishoda su odgovori: poslužitelj je zahtjev primio i rekao što se dogodilo. Treći ishod nije odgovor nego njegov izostanak. Zahtjev se možda izgubio, možda se još obrađuje, a možda je obrađen i izgubio se odgovor. Kod mora za svaki od tih slučajeva imati smislen nastavak, a korisniku pošteno reći da ishod nije poznat.

4 RPC: udaljeni poziv koji izgleda lokalno

Pisati `fetch`, slagati JSON i čitati statusne kodove za svaki poziv između usluga je zamorno. Zato postoje okviri koji udaljeni poziv skrivaju iza obične funkcije.

Definicija 3.2

Poziv udaljene procedure (remote procedure call, RPC) omogućuje programu da pozove funkciju koja se izvršava u drugom procesu, obično na drugom računalu, kao da je lokalna. RPC okvir pretvara argumente u poruku, šalje je, a iz poruke odgovora vraća rezultat.

Kleppmann opisuje mehanizam ovako [1, odjeljak 1.3]. Na mjestu udaljene funkcije RPC okvir daje pozivatelju **stub**: lokalnu funkciju s istim potpisom kao prava funkcija. Stub ne izvršava pravi posao. On argumente pretvara u poruku i šalje je udaljenom čvoru. Na drugoj strani RPC okvir poruku pretvara natrag u argumente i poziva pravu funkciju. Rezultat putuje istim putem natrag, a stub ga vraća pozivatelju (Figure 1).

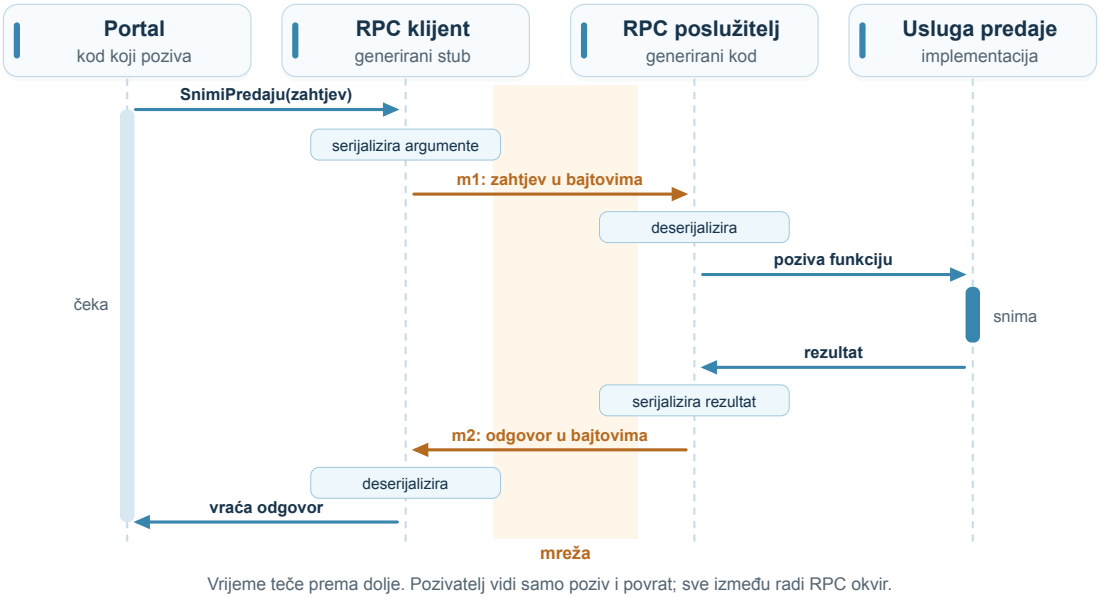


Figure 1: Jedan RPC poziv. Pozivatelj vidi samo poziv i povrat; serijalizaciju, slanje i deserijalizaciju obavlja RPC okvir. Vrijeme teče prema dolje. Prema Kleppmannovu prikazu [1, slajd 16], prilagođeno primjeru portala.

Cilj RPC-a je **lokacijska transparentnost** (location transparency): sustav skriva gdje se resurs nalazi [1, odjeljak 1.3]. Pozivatelj piše `predaja.SnimiPredaju(zahtjev)` kao da je funkcija u istom programu. Upravo tu je zamka. Tablica s početka poglavlja pokazuje da se udaljeni poziv ponaša drukčije, a ta se razlika ne može sakriti. Što ako se usluga sruši tijekom poziva? Što ako se poruka izgubi ili zakasni? Je li sigurno pokušati ponovno?

[1, odjeljak 1.3] Dobar RPC okvir zato ne skriva sve: traži rok čekanja i vraća posebnu pogrešku kada ishod nije poznat.

4.1 gRPC i ugovor sučelja

RPC okviri postoje od 1980-ih: Sun RPC, CORBA, Java RMI, SOAP, Thrift i gRPC (Google, 2015) [1, odjeljak 1.3]. Kada usluge pišu različiti timovi u različitim jezicima, potreban im je zajednički opis sučelja neovisan o jeziku. Takav opis piše se u **jeziku za opis sučelja** (interface definition language, IDL), a iz njega se automatski generira kod za stubove i serijalizaciju [1, odjeljak 1.3].

gRPC kao IDL koristi Protocol Buffers. Ugovor usluge predaje zapisujemo u datoteci `predaja.proto`:

```
syntax = "proto3";
package predaja;

service Predaja {
  rpc SnimiPredaju(ZahtjevPredaje) returns (OdgovorPredaje);
}

message ZahtjevPredaje {
  string predaja_id = 1;      // isti u svakom pokušaju
  string zadatak = 2;
  int64 velicina_bajtova = 3;
}

message OdgovorPredaje {
  string zapis_id = 1;
  bool vec_snimljeno = 2;     // true: ponovljeni pokušaj
}
```

Iz datoteke `.proto` prevoditelj (`protoc`) s dodacima za gRPC generira kod klijenta i poslužitelja za odabrani jezik [2]. Ovo je **unarni** RPC: klijent šalje jedan zahtjev i dobiva jedan odgovor, „baš kao običan poziv funkcije” [2]. gRPC podržava i tokove poruka, ali njih ovdje ne obrađujemo.

U Pythonu poziv iz portala izgleda ovako:

```
import grpc
import predaja_pb2, predaja_pb2_grpc       # generirano iz predaja.proto

kanal = grpc.insecure_channel("predaja:50051")
stub = predaja_pb2_grpc.PredajaStub(kanal)
zahtjev = predaja_pb2.ZahtjevPredaje(
    predaja_id="p-2026-0812", zadatak="RS-projekt",
```

```
    velicina_bajtova=20_971_520)

try:
    odgovor = stub.SnimiPredaju(zahtjev, timeout=2.0)
except grpc.RpcError as e:
    if e.code() == grpc.StatusCode.DEADLINE_EXCEEDED:
        ...   # ishod nepoznat: predaja je možda snimljena
```

Poziv `stub.SnimiPredaju(...)` izgleda lokalno, ali argument `timeout` i iznimka `DEADLINE_EXCEEDED` otkrivaju da nije. Ugovor i kod u ovom poglavlju isprobani su s gRPC-om za Python (grpcio 1.63); primjer za samostalno pokretanje nalazi se u mapi `assets/kod/`.

5 Serijalizacija

Definicija 3.3

Serijalizacija (serialization, marshalling) pretvara podatke iz memorije programa u niz bajtova koji se može poslati mrežom ili spremiti. **Deserijalizacija** je obrnuti postupak.

Isti zahtjev za snimanje predaje možemo serijalizirati na dva načina. U JSON-u je to tekst od 79 bajtova kada se zapiše u jednom retku, bez razmaka; ovdje je prelomljen radi čitljivosti:

```
{"predaja_id":"p-2026-0812","zadatak":"RS-projekt",
 "velicina_bajtova":20971520}
```

U formatu Protocol Buffers isti sadržaj zauzima 30 bajtova:

```
0a 0b 70 2d 32 30 32 36 2d 30 38 31 32   polje 1, duljina 11: "p-2026-0812"
12 0a 52 53 2d 70 72 6f 6a 65 6b 74      polje 2, duljina 10: "RS-projekt"
18 80 80 80 0a                           polje 3, broj 20971520
```

Razlika ne dolazi samo od kraćeg zapisa brojeva. JSON u svakoj poruci ponavlja nazive polja, pa ga čovjek može pročitati i bez ugovora. Protocol Buffers u poruku ne upisuje nazive, nego samo **brojeve polja** iz ugovora; naziv i tip polja primatelj saznaje tek iz definicije poruke [6]. Poruka je zato manja i brža za obradu, ali bez datoteke `.proto` teško ju je razumjeti.

	JSON	**Protocol Buffers**
Zapis	tekst, čitljiv čovjeku	binarni
Polja u poruci	nazivi	brojevi iz ugovora
Ugovor	nije obavezan (može se dodati, npr. OpenAPI)	obavezan (`.proto`)
Veličina primjera	79 bajtova	30 bajtova
Tipično	preglednik poslužitelj, javni API	usluga usluga unutar sustava

Brojke vrijede za ovaj primjer, a ne općenito. Kod velikih tekstualnih polja razlika je manja jer sam tekst zauzima jednako mnogo mjesta u oba formata.

5.1 Promjena ugovora bez rušenja klijenata

Usluge se izdaju neovisno (predavanje 02). Zato se ugovor mijenja dok stari klijenti još rade: usluga predaje može dobiti novu verziju danas, a portal tek sljedeći tjedan. Promjena mora biti **kompatibilna** u oba smjera.

Pretpostavimo da zahtjevu dodajemo polje `string komentar = 4;` za studentov komentar uz predaju.

- **Stari klijent, novi poslužitelj:** stari klijent ne šalje polje 4. Polje koje nije postavljeno ne šalje se mrežom, a primatelj za njega dobiva zadanu vrijednost [5], ovdje prazan niz. Novi poslužitelj mora prazan niz razumjeti kao „bez komentara”.
- **Novi klijent, stari poslužitelj:** stari poslužitelj ne poznaje polje 4. Poruke u formatu proto3 čuvaju takva **nepoznata polja** pri parsiranju i uključuju ih u ponovno serijaliziranu poruku [5], pa poslužitelj neće pasti.

Brojevi polja su ono što se ne smije dirati. Broj polja „ne može se promijeniti nakon što je vrsta poruke u upotrebi” jer identificira polje u zapisu poruke, a brojeve polja „nikada ne treba ponovno koristiti” [5]. Kada polje uklonimo, njegov broj treba **rezervirati** naredbom `reserved 2;`, kako ga netko kasnije slučajno ne bi dodijelio novom polju [7]. Inače bi stari klijent poslao vrijednost starog polja pod brojem koji novi poslužitelj tumači kao nešto sasvim drugo.

Usporedba s JSON-om je poučna. U JSON-u je preimenovanje polja nekompatibilna promjena jer se u poruci šalje naziv. U Protocol Buffersu preimenovanje ne mijenja poruku, ali promjena broja polja je nekompatibilna.

6 Rok čekanja

U predavanju 01 vidjeli smo tri povijesti koje klijentu izgledaju jednako: zahtjev se izgubio, poslužitelj se srušio ili se izgubio odgovor. Za `SnimiPredaju` to znači:

Što se dogodilo	**Je li predaja snimljena?**
zahtjev se izgubio	ne
snimanje je sporo i još traje	da, uskoro
snimanje je gotovo, odgovor se izgubio	da

Ponoviti poziv bez razmišljanja sigurno je samo u prvom retku, a upravo njega klijent ne može razlikovati od druga dva.

Definicija 3.4

Rok (deadline) je trenutak nakon kojega klijent više ne čeka odgovor. **Istek vremena čekanja (timeout)** je trajanje od slanja zahtjeva do tog trenutka.

Razlika je mala, ali korisna: istek čekanja kaže „čekaj 2 sekunde”, a rok „čekaj do 10:00:05,000”. Rok se može proslijediti dalje kroz lanac usluga, a trajanje ne može.

6.1 Rok u gRPC-u

gRPC sam ne postavlja rok: bez njega klijent može čekati odgovor praktički zauvijek. Dokumentacija zato preporučuje da klijent uvijek izričito postavi realan rok [3]. Kada rok prođe, klijent odustaje i poziv završava pogreškom `DEADLINE_EXCEEDED` [3]. Poslužitelj može provjeravati je li poziv otkazan, ali posao koji je već napravio time se ne poništava.

To potvrđuje i kratak pokus s generiranim kodom iz ovog poglavlja: kada je snimanje na poslužitelju trajalo 3 s, a klijent čekao 2 s, klijent je dobio `DEADLINE_EXCEEDED`, a poslužitelj je predaju ipak snimio. Dokumentacija gRPC-a to kaže izravno: klijent i poslužitelj neovisno i lokalno zaključuju je li poziv uspio, i njihovi se zaključci ne moraju podudarati [2].

6.2 Kako odabrati rok

Prekratak rok proglašava spore, ali ispravne pozive neuspjelima i izaziva nepotrebne ponovne pokušaje. Predug rok drži korisnika i resurse u čekanju. Brooker preporučuje da se krene od izmjerenih latencija usluge koju pozivamo i odabere visoki percentil, na primjer p99,9 [11]: tada će otprilike jedan od tisuću ispravnih poziva lažno isteći. Rok je odluka o kompromisu, a ne svojstvo mreže.

6.3 Prijenos roka kroz lanac

Korisnički zahtjev često prolazi kroz nekoliko usluga (predavanje 02). Ako svaka usluga za svoj sljedeći poziv izmisli novi istek čekanja, usluge iza preglednika mogu još dugo raditi nakon što je preglednik odustao. Taj posao nitko ne čeka, a troši resurse upravo kada je sustav opterećen.



Figure 2: Gore: svaka usluga bira vlastiti istek čekanja, pa posao u lancu traje i nakon što je preglednik odustao. Dolje: svi pozivi dijele isti apsolutni rok, a svaki skok troši dio preostalog vremena. Vremena su ilustrativna.

Googleova knjiga o pouzdanosti sustava zato preporučuje **prijenos roka** (deadline propagation): umjesto da izmišlja rok, poslužitelj ga prenosi u svoje pozive, pa svi pozivi u stablu koje je pokrenuo jedan zahtjev imaju isti apsolutni rok [12]. Neke implementacije gRPC-a rok iz dolaznog poziva automatski prenose u odlazni [3]. Usluga koja primi zahtjev kojemu je rok već prošao ne treba ni započeti posao.

7 Ponovni pokušaji

Kada rok istekne, klijent može odustati ili pokušati ponovno. Izbor određuje koje **jamstvo** dajemo o broju izvršavanja operacije.

Jamstvo	**Koliko se puta operacija izvrši**	**Kako se postiže**
najviše jednom (at-most-once)	0 ili 1	bez ponavljanja ili tako da poslužitelj prepozna ponovljeni zahtjev i ne izvrši ga ponovno
barem jednom (at-least-once)	1 ili više, ako klijent primi odgovor	klijent ponavlja dok ne dobije potvrdu

Jamstvo	Koliko se puta operacija izvrši	Kako se postiže
učinak točno jednom (exactly-once)	možda više puta, ali učinak je kao da je jednom	klijent ponavlja, a operacija je idempotentna ili poslužitelj prepoznaje duplikate

Kleppmann jamstva opisuje preko ponašanja klijenta: najviše jednom znači poslati zahtjev i ne ponavljati, barem jednom ponavljati do potvrde, a točno jednom ponavljati uz idempotentnost ili otkrivanje duplikata [1, odjeljak 5.1]. Jamstvo *najviše jednom* može se, međutim, postići i uz ponavljanje. Specifikacija ONC RPC-a opisuje poslužitelj koji pamti već obrađene zahtjeve prema identifikatoru transakcije koji nosi svaka poruka, kako bi osigurao „određeni stupanj" izvršavanja najviše jednom; klijent pri ponovnom slanju smije koristiti isti identifikator [13, odjeljak 5]. Isto radi ključ idempotentnosti koji uvodimo niže.

Zato je korisno jamstva složiti ovako: **učinak točno jednom = barem jednom + najviše jednom**. Klijent ponavlja dok ne dobije odgovor, a poslužitelj ponovljeni zahtjev ne izvršava ponovno ili ga izvršava bez dodatnog učinka. Isti RFC kaže i što klijent zna: ako primi odgovor, procedura je izvršena barem jednom, a ako ne primi odgovor, ne može pretpostaviti da procedura nije izvršena [13, odjeljak 5]. Naziv „točno jednom" zato treba čitati pažljivo. Mreža ne može jamčiti da će poruka biti obrađena točno jednom; možemo postići samo da ponovljena obrada nema dodatni učinak.

7.1 Ponovni pokušaji se množe

U predavanju 01 vidjeli smo da ponovni pokušaji mogu preopteretiti uslugu kojoj trebaju pomoći. Problem raste kada ponavlja više slojeva. U Googleovu primjeru JavaScript u pregledniku, frontend i backend svaki ponavljaju do tri puta, odnosno šalju do četiri pokušaja, pa jedna radnja korisnika može izazvati $4^3 = 64$ pokušaja na bazi [12]. Brooker opisuje isti učinak: ponovni pokušaji na svakom sloju se množe [11].

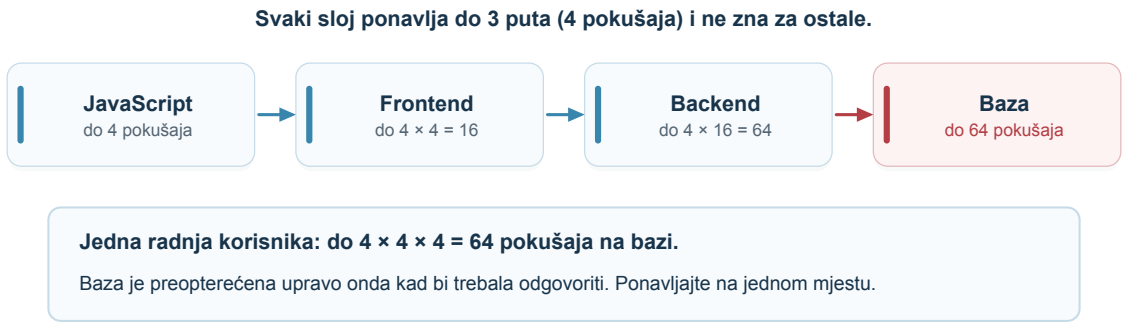


Figure 3: Svaki sloj šalje do četiri pokušaja i ne zna za ostale. Na bazi to daje do 64 pokušaja za jednu radnju korisnika [12].

Pravilo je jednostavno: ponavljajte na jednom mjestu, ograničite broj pokušaja i između njih pričekajte, s rastućim razmakom i nasumičnim pomakom (predavanja 01 i 08).

7.2 Ponovni pokušaj u gRPC-u

gRPC može sam ponavljati pozive, ali samo prema **politici ponovnih pokušaja** koju zadamo za svaku metodu posebno [4]. Bez takve politike gRPC u većini slučajeva ne može sigurno ponoviti poziv [4]. Politika se zadaje u konfiguraciji usluge:

```
{"methodConfig": [{
  "name": [{"service": "predaja.Predaja"}],
  "retryPolicy": {
    "maxAttempts": 4,
    "initialBackoff": "0.1s", "maxBackoff": "1s", "backoffMultiplier": 2,
    "retryableStatusCodes": ["UNAVAILABLE"]
  }
}]}
```

Dvije pojedinosti su važne. Prvo, kada klijent primi zaglavlje odgovora, poziv je „potvrđen” i gRPC ga više ne ponavlja [4]. Drugo, politika ponavljanja tvrdnja je programera da je metodu sigurno ponoviti. Kod `UNAVAILABLE` obično znači da usluga nije bila dostupna, ali veza se može prekinuti i nakon što je poslužitelj već obavio posao. Politika ponovnih pokušaja ne čini metodu sigurnom; metoda mora biti sigurna sama po sebi.

8 Idempotentnost

Definicija 3.5

Operacija je **idempotentna** (idempotent) ako višestruko izvođenje ima isti učinak kao jedno izvođenje. Odgovori se pritom mogu razlikovati.

Matematički, funkcija f je idempotentna ako je $f(f(x)) = f(x)$ [1, odjeljak 5.1]. Kleppmannov primjer je gumb „sviđa mi se”. Operacija „povećaj brojač za jedan” nije idempotentna: ponovljeni zahtjev dodaje još jedan glas. Operacija „dodaj korisnika u skup korisnika kojima se sviđa” jest idempotentna: korisnik je u skupu nakon jednog ili nakon deset pokušaja, a broj glasova računa se kao veličina skupa [1, odjeljak 5.1].

HTTP poznaje isto svojstvo. Metoda je idempotentna ako je namjeravani učinak više jednakih zahtjeva na poslužitelju jednak učinku jednog takvog zahtjeva; od metoda definiranih u standardu idempotentne su PUT, DELETE i sigurne metode, one koje samo čitaju, poput GET-a [8, odjeljak 9.2]. Zato klijent smije, kada se veza prekine prije odgovora, otvoriti novu vezu i ponoviti idempotentan zahtjev [8, odjeljak 9.2.2]. POST nije idempotentan, a predaja projekta je upravo POST.

Mnoge operacije možemo preoblikovati da postanu idempotentne:

Nije idempotentno	Idempotentno
povećaj brojač glasova	dodaj korisnika u skup glasova
dodaj 10 bodova	postavi bodove na 85
snimi još jedan zapis predaje	snimi predaju `p-2026-0812`, ako već nije snimljena
stvori novu predaju s novim identifikatorom	snimi predaju pod identifikatorom koji je klijent unaprijed odabrao

Posljednja dva retka pokazuju opći trik: operaciji damo **identitet** koji ostaje isti u svim pokušajima.

8.1 Ključ idempotentnosti

> **Definicija 3.6**
>
> **Ključ idempotentnosti (idempotency key)** je jedinstvena vrijednost koju klijent generira za jednu namjeravanu operaciju i šalje je u svakom njezinu pokušaju. Poslužitelj po ključu prepoznaje ponovljene pokušaje iste operacije.

Definicija slijedi prijedlog standarda IETF-a za HTTP zaglavlje `Idempotency-Key` [10]. To je zasad nacrt (Internet-Draft), a ne usvojeni standard, ali isti obrazac godinama koriste javni API-ji. U pregledniku to izgleda ovako:

```
const kljuc = crypto.randomUUID();   // jednom, prije prvog pokušaja

function posaljiPredaju() {           // isti ključ u svakom pokušaju
  return fetch("/predaje", {
    method: "POST", body: zip,
    headers: { "Idempotency-Key": kljuc },
    signal: AbortSignal.timeout(30_000),
  });
}
```

Stripe, usluga za online plaćanja, ponašanje opisuje precizno [9]. Poslužitelj sprema statusni kod i tijelo odgovora prvog zahtjeva s određenim ključem, a ponovljeni zahtjevi s istim ključem dobivaju taj spremljeni odgovor, čak i ako je to bila pogreška. Poslužitelj uspoređuje parametre ponovljenog zahtjeva s izvornima i javlja pogrešku ako se razlikuju. Ključevi se smiju brisati kada su stari barem 24 sata.

Iz toga slijede pravila za klijenta i poslužitelj:

- Ključ se generira **jednom za namjeravanu operaciju**, a ne za svaki pokušaj. Novi ključ u svakom pokušaju poslužitelju izgleda kao nova predaja.
- Isti ključ s drukčijim sadržajem je pogreška, a ne nova operacija.
- Ključ se čuva dovoljno dugo da pokrije sve razumne ponovne pokušaje.

U usluzi predaje ključ već imamo: `predaja_id`. Portal ga odabere jednom i šalje u svakom pokušaju, a usluga predaje snima najviše jedan zapis po predaji.

8.2 Ključ i učinak spremaju se zajedno

Ključ pomaže samo ako ga poslužitelj sprema **atomarno** s učinkom. Pretpostavimo da poslužitelj najprije snimi predaju, a zatim u zasebnom koraku spremi ključ. Ako se sruši između ta dva koraka, predaja je snimljena, ali ključ nije zapisan. Ponovljeni pokušaj s istim ključem tada izgleda kao nova operacija i snima drugi zapis iste predaje. U modelu s oporavkom nakon ispada otkrivanje duplikata zato traži da se zahtjevi ili podaci o njima spremaju u trajnu pohranu [1, odjeljak 5.1].



Figure 4: Lijevo: učinak i ključ spremaju se u dva koraka; rušenje između njih ostavlja učinak bez ključa, pa ponovni pokušaj stvara duplikat. Desno: ključ, učinak i odgovor u jednoj transakciji; nakon rušenja ponovni pokušaj dobiva spremljeni odgovor.

U relacijskoj bazi najjednostavnije je da ključ bude jedinstven stupac u istoj tablici u kojoj nastaje učinak. U usluzi predaje to je stupac `predaja_id` s ograničenjem `UNIQUE`:

```
INSERT INTO predaje (zapis_id, predaja_id, zadatak) VALUES ($1, $2, $3)
ON CONFLICT (predaja_id) DO NOTHING
RETURNING zapis_id;
```

Ako naredba ne vrati redak, zapis te predaje već postoji; poslužitelj ga pročita i vrati isti `zapis_id` s oznakom `vec_snimljeno = true`. Ograničenje `UNIQUE` važno je i kada dva pokušaja stignu istodobno. Kao u primjeru etcd-a iz predavanja 01, odvojeno „pročitaj pa upiši" nije dovoljno: oba pokušaja mogu pročitati da zapisa nema i oba ga snimiti. Jedna atomarna naredba to ne dopušta.

Ostaje jedna granica. Transakcija u bazi obuhvaća samo bazu. Ako usluga u istom koraku pošalje i e-poštu, e-pošta je vanjski učinak koji se ne može poništiti. Kako takve učinke povezati s transakcijom, obrađuju predavanja 09 i 10.

9 Natrag na početak: smije li portal ponoviti poziv?

Odgovor je „ovisi", a sada znamo o čemu:

- **Bez ključa:** ne smije bez razmišljanja. Ako se izgubio odgovor, ponovni pokušaj snima drugi zapis iste predaje.
- **S ključem `predaja_id` i atomarnim spremanjem:** smije. Ponovljeni pokušaj ne snima novi zapis, nego vraća postojeći.
- **U svakom slučaju:** rok za svaki pokušaj, ograničen broj pokušaja i razmak između njih, ponavljanje na jednom sloju te prijenos roka.

Tako je i studentu: preglednik koji predaju šalje s ključem idempotentnosti može nakon isteka čekanja pokušati ponovno, a portal neće stvoriti dvije predaje.

10 Kako napraviti loš udaljeni poziv

- Poziv bez roka: klijent može čekati zauvijek.
- Ponavljanje na svakom sloju: pokušaji se množe.
- Ponavljanje POST-a ili druge neidempotentne operacije bez ključa.
- Novi ključ idempotentnosti u svakom pokušaju.
- Spremanje ključa u zasebnom koraku nakon učinka.
- Promjena ili ponovna upotreba broja polja u datoteci `.proto`.
- Udaljeni poziv u petlji za svaki element popisa (razgovorljivo sučelje iz predavanja 02).

11 Zadaci i rasprava

1. **Lokalno ili udaljeno.** Kolega tvrdi: „gRPC poziv je samo poziv funkcije, kod ne treba mijenjati." Navedite tri razlike koje kod mora uzeti u obzir i za svaku napišite što bi kod trebao napraviti.
2. **Serijalizacija.** Poruka ima polja `ime` (string), `bodovi` (cijeli broj) i `predano` (logička vrijednost). Napišite je kao JSON i kao poruku u datoteci `.proto`. Koja promjena ugovora je kompatibilna u JSON-u, a nije u Protocol Buffersu, i obrnuto?

3. **Ugovor se mijenja.** Iz poruke `ZahtjevPredaje` uklanjamo polje `zadatak = 2`, a kasnije dodajemo novo polje `string rok = 2;`. Što se događa kada stari klijent pošalje zahtjev novom poslužitelju? Kako je to trebalo spriječiti?
4. **Rok u lancu.** Preglednik čeka 5 s. Portal svom pozivu usluzi predaje postavlja istek čekanja od 5 s, a usluga predaje pohrani datoteka također 5 s. Opišite što se događa ako je pohrana spora. Kako bi pomogao prijenos roka?
5. **Semantike.** Za svaku operaciju odaberite semantiku ponovnih pokušaja i obrazložite: (a) slanje metrike o opterećenju procesora svakih 10 s; (b) upis ocjene u sustav; (c) obavijest nastavniku o novoj predaji.
6. **Idempotentno ili ne.** Koje su operacije idempotentne? Preoblikujte one koje nisu: (a) `DELETE /predaje/812`; (b) `POST /predaje/812/komentari`; (c) „povećaj broj predaja za 1”; (d) `PUT /predaje/812/ocjena` s tijelom `{"bodovi": 85}`.

11.1 Predloženo rješenje

1. Udaljeni poziv može trajati dugo, pa treba rok. Može završiti bez odgovora, pa kod mora imati granu za nepoznat ishod i korisniku to pošteno reći. Usluga može biti nedostupna ili spora neovisno o pozivatelju, pa treba odlučiti hoće li se poziv ponoviti i je li to sigurno. Argumenti se kopiraju: promjena objekta nakon poziva ne mijenja ono što je poslužitelj primio.
2. JSON: `{"ime": "Ana", "bodovi": 85, "predano": true}`. Protocol Buffers: `message Rezultat { string ime = 1; int32 bodovi = 2; bool predano = 3; }`. Preimenovanje polja je nekompatibilno u JSON-u, a kompatibilno u Protocol Buffersu jer se naziv ne šalje. Promjena broja polja nekompatibilna je u Protocol Buffersu, a u JSON-u brojeva polja nema.
3. Stari klijent i dalje šalje naziv zadatka pod brojem 2. Novi poslužitelj tu vrijednost čita kao rok i dobiva besmislen podatak, bez ikakve pogreške. Broj 2 trebalo je rezervirati (`reserved 2;`) i novom polju dati novi broj.
4. Preglednik nakon 5 s odustane, ali portal još čeka do 5 s na uslugu predaje, a usluga predaje do 5 s na pohranu. Posao se nastavlja iako ga nitko ne čeka, a sporu pohranu dodatno opterećuju ponovni pokušaji. S prijenosom roka svi pozivi imaju isti apsolutni rok; usluga predaje vidi da je preostalo malo vremena, a kada rok prođe, posao se prekida ili ne započinje.
5. (a) Najviše jednom: izgubljena metrika nije važna, a sljedeća stiže za 10 s. (b) Učinak točno jednom: ponavljati s ključem ili idempotentnom operacijom „postavi ocjenu”. (c) Barem jednom ili učinak točno jednom: dvostruka e-pošta je neugodna, a izgubljena obavijest može biti važnija; s identifikatorom predaje usluga obavijesti može prepoznati duplikat.
6. (a) Idempotentno: nakon prvog brisanja predaje nema; drugi zahtjev može dobiti odgovor 404, ali učinak je isti. (b) Nije: svaki pokušaj dodaje komentar; rješenje je ključ idempotentnosti ili identifikator komentara koji odabire klijent (`PUT /predaje/812/komentari/{id}`). (c) Nije: predaje treba spremati u skup s jedinstvenim identifikatorom, a broj računati iz skupa. (d) Idempotentno: postavlja vrijednost umjesto da je mijenja za iznos.

12 Sažetak

- Udaljeni poziv ima tri ishoda: uspjeh, pogrešku i nepoznat ishod. Treći se ne može sakriti.
- RPC poziv udaljene funkcije izgleda lokalno: stub serijalizira argumente, a RPC okvir šalje poruku i vraća rezultat. gRPC ugovor piše u datoteci `.proto` i iz nje generira kod.
- Protocol Buffers u poruku upisuje brojeve polja, ne nazive. Brojevi se ne smiju mijenjati ni ponovno koristiti; uklonjeni se rezerviraju.
- Svaki poziv treba rok. Rok se prenosi kroz lanac usluga, a ne izmišlja na svakom skoku.
- Ponovni pokušaj nakon isteka čekanja daje semantiku *barem jednom*. Ponavljanje na više slojeva množi pokušaje.
- *Učinak točno jednom* postižemo ponavljanjem uz idempotentnu operaciju ili ključ idempotentnosti, spremljen atomarno s učinkom.

12.1 Osmislite sljedeći korak: dva uređaja i e-pošta

Studentica počne predaju na laptopu i, jer se ništa ne događa, isti ZIP pošalje i s mobitela. Svaki uređaj generira vlastiti ključ idempotentnosti. Hoće li portal stvoriti dvije predaje? Što bi bio bolji ključ? Drugi slučaj: usluga predaje pošalje nastavniku e-poštu, a zatim se sruši prije nego što spremi ključ i zapis. Što se događa pri ponovnom pokušaju i zašto transakcija u bazi tu ne pomaže?

13 Izvori i dodatna literatura

1. M. Kleppmann, *Distributed Systems*, bilješke s predavanja, Sveučilište u Cambridgeu, 2021./22., odjeljci 1.3 „Example: Remote Procedure Calls (RPC)” (str. 8–11) i 5.1 „Manipulating remote state” (str. 47–48). https://www.cl.cam.ac.uk/teaching/2122/ConcDisSys/dist-sys-notes.pdf
2. gRPC, „Core concepts, architecture and lifecycle”. https://grpc.io/docs/what-is-grpc/core-concepts/
3. gRPC, „Deadlines”. https://grpc.io/docs/guides/deadlines/
4. gRPC, „Retry”. https://grpc.io/docs/guides/retry/
5. Protocol Buffers, „Language Guide (proto 3)”. https://protobuf.dev/programming-guides/proto3/
6. Protocol Buffers, „Encoding”. https://protobuf.dev/programming-guides/encoding/
7. Protocol Buffers, „Proto Best Practices”. https://protobuf.dev/best-practices/dos-donts/
8. R. Fielding, M. Nottingham i J. Reschke (ur.), „HTTP Semantics”, RFC 9110, lipanj 2022., odjeljak 9.2. https://www.rfc-editor.org/rfc/rfc9110.html
9. Stripe, „Idempotent requests”, API dokumentacija. https://docs.stripe.com/api/idempotent_requests

10. J. Jena, S. Dalal i A. Tishkin, „The Idempotency-Key HTTP Header Field”, Internet-Draft draft-ietf-httpapi-idempotency-key-header-08, IETF, 2026. https://datatracker.ietf.org/doc/draft-ietf-httpapi-idempotency-key-header/
11. M. Brooker, „Timeouts, retries, and backoff with jitter”, Amazon Builders' Library, 2019. https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/
12. M. Ulrich, „Addressing Cascading Failures”, u: B. Beyer i dr. (ur.), *Site Reliability Engineering*, O'Reilly, 2016., pogl. 22. https://sre.google/sre-book/addressing-cascading-failures/
13. R. Thurlow, „RPC: Remote Procedure Call Protocol Specification Version 2”, RFC 5531, svibanj 2009., odjeljak 5. https://www.rfc-editor.org/rfc/rfc5531.html

Dodatno: J. Waldo, G. Wyant, A. Wollrath i S. Kendall, „A Note on Distributed Computing”, Sun Microsystems Laboratories, 1994., klasičan rad o tome zašto se udaljeni objekti ne mogu ponašati kao lokalni (navedeno prema [1]).